

Epicor Interview — Complete Revision Guide

Product Developer (.NET, Angular, SQL) — Round 1, 21 July 2026

Epicor — Product Developer (.NET, Angular, SQL) — Deep Prep Guide

Round 1 (Teams) — 21 July 2026, 12 PM

This guide is built directly against the JD you shared: C#, .NET 8+, ASP.NET Core, Angular 15+, EF Core, SQL Server, architecture patterns, testing (xUnit/Moq), Azure, Docker/K8s basics, auth (OAuth2/JWT/OIDC), and Agile. Given you're 3 years in against a 3–5 year band, the interviewer will likely probe depth over breadth — expect “why,” not just “what.” Answer with a reason + a tiny example every time.

1. C# / .NET Core Fundamentals

Q1. What's the difference between .NET Framework, .NET Core, and .NET 8? .NET Framework is Windows-only, legacy, no longer under active feature development. .NET Core (now unified as just ".NET" from .NET 5 onward) is cross-platform, open source, faster, modular via NuGet. .NET 8 is an LTS (Long Term Support) release — 3 years of support — vs. non-LTS releases (like .NET 7) that get 18 months. Say: "We run production on .NET 8 for the LTS guarantee."

Q2. Value types vs reference types? Value types (`struct`, `int`, `bool`, `enum`) live on the stack (or inline in containing objects) and are copied on assignment. Reference types (`class`, `string`, arrays) live on the heap; variables hold a pointer/reference. `string` is a reference type but *behaves* immutably — every "modification" creates a new string object.

Bug-spotting example:

```
struct Point { public int X, Y; }
void Modify(Point p) { p.X = 100; }

var pt = new Point { X = 1, Y = 2 };
Modify(pt);
Console.WriteLine(pt.X); // prints 1, not 100 – struct passed by value (copy)
```

Fix: pass `ref Point p` if mutation is intended, or better, avoid mutable structs entirely.

Q3. readonly vs const? `const` is a compile-time constant, must be assigned at declaration, implicitly static. `readonly` is assigned at declaration or in the constructor, evaluated at runtime, can differ per instance.

Q4. Boxing and unboxing — why does it matter for performance? Boxing wraps a value type in an object on the heap (implicit); unboxing extracts it back (explicit cast). Costs an allocation + GC pressure. Common silent-boxing trap:

```
int x = 5;
object o = x; // boxing
ArrayList list = new ArrayList();
list.Add(x); // boxing again – ArrayList stores object
```

Modern fix: use generic collections (`List<int>`) which avoid boxing entirely.

Q5. IEnumerable vs IQueryable — this comes up constantly with EF Core. `IEnumerable<T>` executes in-memory (LINQ to Objects) — the whole sequence is already loaded, filtering happens client-side. `IQueryable<T>` builds an **expression tree** that's translated into SQL and executed at the database, only when enumerated (deferred execution). Mixing them wrong is a classic performance bug:

```
// BAD: pulls entire Orders table into memory before filtering
IEnumerable<Order> orders = _context.Orders;
var recent = orders.Where(o => o.Date > DateTime.Now.AddDays(-7));

// GOOD: filter translated to SQL WHERE clause, only matching rows come back
IQueryable<Order> orders = _context.Orders;
var recent = orders.Where(o => o.Date > DateTime.Now.AddDays(-7));
```

Q6. Explain deferred execution and give a bug caused by it. LINQ queries (`IQueryable`/`IEnumerable`, not already materialized with `.ToList()` / `.ToArray()` / `.Count()` / `.First()`) don't run until enumerated. Classic bug — closure captures a variable by reference, not value:

```
int threshold = 10;
var query = orders.Where(o => o.Total > threshold);
threshold = 100; // this DOES change query results because it hasn't executed yet!
foreach (var o in query) { ... } // uses threshold=100, not 10
```

Also the "modified collection" bug — iterating and mutating the same `List<T>` throws `InvalidOperationException` because deferred LINQ re-reads the source at enumeration time.

Q7. async/await — what problem does it solve, and what's a common deadlock bug? It frees the thread during I/O-bound waits (DB calls, HTTP calls) instead of blocking it, improving throughput under load. Classic ASP.NET (pre-Core) deadlock: blocking on async code with `.Result` or `.Wait()` captures the `SynchronizationContext`, and when the continuation tries to resume on that same captured context, it can't — because the context is busy waiting on `.Result`.

```
// Deadlock-prone (in a context with a SynchronizationContext, e.g. classic ASP.NET or UI apps)
public ActionResult Get()
{
    var data = GetDataAsync().Result; // blocks + deadlocks
    return View(data);
}
```

Fix: `async` all the way down (`await GetDataAsync()`), or use `ConfigureAwait(false)` in library code to avoid re-marshalling to the

original context. Note: ASP.NET Core has no `SynchronizationContext`, so this specific deadlock is less likely there — but blocking async code is still a bad practice (thread starvation under load).

Q8. Task.Run vs async/await — when to use which? `async/await` is for I/O-bound work (DB, network) — no extra thread needed, the thread is released while waiting. `Task.Run` schedules CPU-bound work onto the thread pool — use it to offload heavy computation from a UI thread, not to make I/O “async” (wrapping an already-sync I/O call in `Task.Run` just hides the blocking, wasting a thread pool thread).

Q9. Dependency Injection — what are the 3 lifetimes and a bug from misusing them? - `Transient`: new instance every time it's requested. - `Scoped`: one instance per HTTP request. - `Singleton`: one instance for app lifetime.

Classic bug — **captive dependency**: injecting a `Scoped` service (e.g., `DbContext`) into a `Singleton` service. The singleton captures the scoped instance on first resolution and holds it forever, causing stale data or cross-request `DbContext` reuse (which is not thread-safe and will throw under concurrent requests).

```
services.AddSingleton<IMyCache, MyCache>(); // singleton
services.AddScoped<AppDbContext>();      // scoped
// If MyCache takes AppDbContext in its constructor -> captive dependency, runtime error or subtle bugs
```

Fix: inject `IServiceScopeFactory` into the singleton and create a scope per operation.

Q10. SOLID — give a one-line example of each, from real production code. - **Single Responsibility**: split a class doing both business logic AND email-sending into two. - **Open/Closed**: adding a new discount type via a new `IDiscountStrategy` implementation instead of an `if/else` chain. - **Liskov Substitution**: a subclass shouldn't throw `NotImplementedException` for a method the base class promises. - **Interface Segregation**: don't force a `ReadOnlyRepository` to implement `Delete()`. - **Dependency Inversion**: controller depends on `IOrderService` interface, not `OrderService` concrete class.

2. ASP.NET Core / Web API / Architecture Patterns

Q11. Explain Clean Architecture / Onion Architecture layering and why Epicor likely cares. Layers, dependencies pointing inward: Domain (entities, no dependencies) → Application (use cases, interfaces) → Infrastructure (EF Core, external services, implements Application interfaces) → Presentation/API (controllers). The JD explicitly lists “Clean Architecture, Repository Pattern” — be ready to sketch this on a whiteboard/share-screen.

Q12. Repository Pattern — pros, cons, and the criticism to be aware of. Pros: abstracts data access, testable via mocking, centralizes query logic. Con (a common interview gotcha): EF Core’s `DbSet<T>` + `DbContext` (with LINQ) is *already* a Unit-of-Work + Repository abstraction — wrapping it in a custom repository can be redundant unless you need to swap ORMs or centralize complex, reused queries. Good answer: “We use repository for aggregate roots where query logic is reused across services, but simple CRUD often just goes through EF Core directly via a thin service layer.”

Q13. CQRS — what is it, and what’s MediatR’s role? Command Query Responsibility Segregation: separate models for reads (queries) and writes (commands) — they can even use different data stores. MediatR is a library implementing the mediator pattern so controllers send `IRequest` objects to handlers without directly depending on service classes — decouples controller from business logic, one handler per use case.

Q14. Design a minimal Web API endpoint with proper layering (write the code).

```
[ApiController]
[Route("api/[controller]")]
public class OrdersController : ControllerBase
{
    private readonly IOrderService _orderService;
    public OrdersController(IOrderService orderService) => _orderService = orderService;

    [HttpGet("{id:int}")]
    [ProducesResponseType(typeof(OrderDto), StatusCodes.Status200OK)]
    [ProducesResponseType(StatusCodes.Status404NotFound)]
    public async Task<IActionResult> GetById(int id, CancellationToken ct)
    {
        var order = await _orderService.GetByIdAsync(id, ct);
        return order is null ? NotFound() : Ok(order);
    }
}
```

Points they may probe: why `CancellationToken`? (propagates client disconnect / request timeout down the async chain, avoiding wasted DB work). Why DTO not entity? (avoid over-posting/leaking internal fields, avoid EF Core lazy-loading serialization issues).

Q15. Middleware pipeline — order matters. What breaks if you put `UseAuthorization()` before `UseAuthentication()`? Authorization checks *who the user is allowed to do* — but that requires Authentication to have already populated `HttpContext.User`. Reversing the order means authorization runs against an unauthenticated/anonymous principal, and every request gets rejected (or worse, silently treated as anonymous-allowed). Correct order: `UseRouting()` → `UseAuthentication()` → `UseAuthorization()` → endpoint.

Q16. What is a custom middleware and when would you write one? (Give code.)

```
public class RequestTimingMiddleware
{
    private readonly RequestDelegate _next;
    public RequestTimingMiddleware(RequestDelegate next) => _next = next;

    public async Task InvokeAsync(HttpContext context)
    {
        var sw = Stopwatch.StartNew();
        await _next(context);
        sw.Stop();
        context.Response.Headers["X-Response-Time-ms"] = sw.ElapsedMilliseconds.ToString();
    }
}

// Registration: app.UseMiddleware<RequestTimingMiddleware>();
```

Real use cases: request logging, global exception handling, correlation IDs, API key validation.

Q17. Global exception handling — the modern .NET 8 way? `IExceptionHandler` (new in .NET 8) registered via `AddExceptionHandler<T>()`, replacing the older `UseExceptionHandler` middleware lambda pattern in many shops. Returns a `ProblemDetails` response consistently.

Q18. Authentication vs Authorization; explain OAuth2, OpenID Connect, and JWT — how do they fit together?

Authentication = “who are you,” Authorization = “what are you allowed to do.” OAuth2 is an *authorization* framework (delegated access via tokens) — it was never designed to prove identity by itself. OpenID Connect is a thin identity layer **on top of** OAuth2 that adds the `id_token` (a JWT) which proves authentication. JWT is just the token *format*: header.payload.signature, base64url-encoded, signed (usually HMAC or RSA) so the API can verify it wasn’t tampered with, without a DB round-trip.

Flow to describe: Angular app redirects to Entra ID (Azure AD) → user logs in → Entra ID issues an `id_token` (who you are) + `access_token` (what you can call) → Angular attaches `Authorization: Bearer <access_token>` on API calls → ASP.NET Core middleware validates the token's signature, issuer, audience, expiry against Entra ID's metadata.

Q19. RBAC — how would you enforce it in ASP.NET Core?

```
[Authorize(Roles = "Admin,Manager")]
[HttpDelete("{id}")]
public async Task<IActionResult> Delete(int id) { ... }

// Or policy-based (more flexible, testable):
services.AddAuthorization(options =>
{
    options.AddPolicy("CanDeleteOrders", policy =>
        policy.RequireClaim("permission", "orders.delete"));
});
[Authorize(Policy = "CanDeleteOrders")]
```

Policy-based is preferred in real systems since it decouples “who can do X” from role names, which change.

3. Entity Framework Core & LINQ

Q20. N+1 query problem — spot the bug and fix it.

```
// BUG: 1 query for customers + N queries for each customer's orders (lazy loading)
var customers = await _context.Customers.ToListAsync();
foreach (var c in customers)
    Console.WriteLine(c.Orders.Count); // triggers a separate DB round-trip per customer

// FIX: eager load with Include
var customers = await _context.Customers
    .Include(c => c.Orders)
    .ToListAsync();
```

Q21. AsNoTracking() — what does it do and when should you use it? Skips EF Core's change tracker (no snapshot kept for detecting modifications) — meaningfully faster for read-only queries (reports, GET endpoints). Never use it if you plan to `.Update()` / `SaveChanges()` on the retrieved entities afterward — changes won't be tracked and won't persist.

Q22. Explain EF Core's change tracking / how `SaveChanges()` knows what SQL to generate. On query, EF Core snapshots original values. On `SaveChanges()`, it compares current entity state vs snapshot, generates `UPDATE` only for changed columns (or `INSERT` / `DELETE` for Added/Deleted states). Entity states: `Added`, `Unchanged`, `Modified`, `Deleted`, `Detached`.

Q23. Explain a subtle “DbContext lifetime” bug in a multi-threaded/async scenario. `DbContext` is **not thread-safe**. Firing two `await` calls on the same context concurrently (e.g., `Task.WhenAll(_context.Orders.ToListAsync(), _context.Customers.ToListAsync())`) throws `InvalidOperationException`: A second operation was started on this context before a previous operation completed. Fix: sequential awaits, or separate scoped contexts per parallel operation.

Q24. Write a LINQ query: get the top 3 customers by total order value, using `GroupBy`.

```
var topCustomers = await _context.Orders
    .GroupBy(o => o.CustomerId)
    .Select(g => new { CustomerId = g.Key, Total = g.Sum(o => o.Amount) })
    .OrderByDescending(x => x.Total)
    .Take(3)
    .ToListAsync();
```

Watch-out to mention: `GroupBy` sometimes doesn't translate efficiently to SQL depending on EF Core version/provider — worth checking generated SQL via logging in performance-sensitive code.

Q25. `Select` vs `SelectMany`? `Select` projects each element 1:1 (nested collections stay nested). `SelectMany` flattens a collection of collections into one sequence.

```
// Select: List<List<int>> stays nested -> IEnumerable<List<int>>
var nested = orders.Select(o => o.Items);
// SelectMany: flattens -> IEnumerable<Item>
var flat = orders.SelectMany(o => o.Items);
```

Q26. Migrations — how do you handle a breaking schema change safely in production? Additive-first approach: add new nullable column → deploy code that writes to both old+new → backfill data → deploy code reading from new column only → drop old column in a later migration. Avoids downtime from a single big-bang migration on a live table.

4. SQL Server (deep — you have real production experience here, lean on it)

Q27. Clustered vs non-clustered index — explain with a concrete example. Clustered index determines the physical row order on disk — a table can have only **one** (usually the PK). Non-clustered index is a separate structure with pointers back to the clustered key (or heap row) — a table can have many. Query `SELECT * FROM Orders WHERE CustomerId = 5` benefits from a non-clustered index on `CustomerId`, but if you `SELECT *`, the engine still needs a **key lookup** back to the clustered index to fetch other columns — that lookup is often the actual cost. Fix: **covering index** — `CREATE INDEX IX_Orders_CustomerId INCLUDE (OrderDate, Amount)` so the query is satisfied entirely from the non-clustered index without a lookup.

Q28. Set-based vs RBAR (row-by-agonizing-row) — give a bug and the fix.

```
-- BAD: RBAR via cursor, slow at scale
DECLARE cur CURSOR FOR SELECT OrderId FROM Orders WHERE Status = 'Pending';
OPEN cur;
FETCH NEXT FROM cur INTO @OrderId;
WHILE @@FETCH_STATUS = 0
BEGIN
    UPDATE Orders SET Status = 'Processed' WHERE OrderId = @OrderId;
    FETCH NEXT FROM cur INTO @OrderId;
END

-- GOOD: single set-based statement, engine optimizes as a whole
UPDATE Orders SET Status = 'Processed' WHERE Status = 'Pending';
```

Q29. Explain SARGability — why does `WHERE YEAR(OrderDate) = 2026` kill index usage? Wrapping a column in a function prevents the optimizer from doing an index seek on that column (it must compute the function for every row = scan). Fix:

```
WHERE OrderDate >= '2026-01-01' AND OrderDate < '2027-01-01'
```

Q30. Write a stored procedure with proper error handling and transaction control.

```
CREATE PROCEDURE dbo.TransferFunds
    @FromAccountId INT,
    @ToAccountId INT,
    @Amount DECIMAL(18,2)
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;

        UPDATE Accounts SET Balance = Balance - @Amount WHERE AccountId = @FromAccountId;
        IF (SELECT Balance FROM Accounts WHERE AccountId = @FromAccountId) < 0
            THROW 50001, 'Insufficient funds', 1;

        UPDATE Accounts SET Balance = Balance + @Amount WHERE AccountId = @ToAccountId;

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF XACT_STATE() <> 0
            ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END
```

Be ready to explain `XACT_STATE()` (checks if there's an active/uncommittable transaction before rolling back — avoids “no transaction to roll back” errors).

Q31. Explain deadlocks in SQL Server, and how to avoid them. Two transactions each hold a lock the other needs. Classic cause: two procedures updating the same tables in **opposite order** (Tx1: Table A then B; Tx2: Table B then A). Fix: always access tables in a **consistent order** across all procedures; keep transactions short; use appropriate isolation level (`READ COMMITTED SNAPSHOT` reduces reader/writer blocking).

Q32. Isolation levels — explain `READ COMMITTED` vs `READ COMMITTED SNAPSHOT` vs `SERIALIZABLE`. `READ COMMITTED` (default): readers block on writers' locks. `READ COMMITTED SNAPSHOT` (RCSI): readers see a versioned snapshot instead of blocking — big concurrency win, common in high-traffic OLTP. `SERIALIZABLE`: strictest, prevents phantom reads, but heavy locking — reserve for critical financial consistency.

Q33. How do you find and fix a slow query in production? `SET STATISTICS IO, TIME ON` + look at the **execution plan** (Actual Execution Plan) — look for Table Scans on large tables, Key Lookups, high “Estimated vs Actual Rows” mismatch (stale statistics → `UPDATE STATISTICS`), missing index suggestions (green hints in SSMS), implicit conversions (data type mismatch between column and parameter kills index seeks).

Q34. Window functions — write a query ranking employees by salary within each department.

```
SELECT EmployeeId, DepartmentId, Salary,  
       RANK() OVER (PARTITION BY DepartmentId ORDER BY Salary DESC) AS DeptRank  
FROM Employees;
```

Know the difference: `ROW_NUMBER()` (always unique, no ties), `RANK()` (ties share rank, gaps after), `DENSE_RANK()` (ties share rank, no gaps).

5. Angular 15+ / TypeScript

Q35. Angular's change detection — default vs OnPush, and a performance bug. Default strategy checks the entire component tree on every event/async callback (dirty checking). `OnPush` only re-renders when an `@Input()` reference changes, an event fires inside the component, or an `Observable` (via `async` pipe) emits. Common bug: mutating an array/object in place (`this.items.push(x)`) under `OnPush` — Angular sees the same reference, doesn't detect the change. Fix: `this.items = [...this.items, x]` (new reference).

Q36. RxJS — explain `switchMap` vs `mergeMap` vs `concatMap`, with a real bug example.

```
// BUG: using mergeMap for a search-as-you-type — old requests can resolve AFTER newer ones,
// showing stale results because responses aren't cancelled, just all run in parallel.
this.searchInput.valueChanges.pipe(
  mergeMap(term => this.api.search(term))
).subscribe(results => this.results = results);

// FIX: switchMap cancels the previous inner observable when a new value arrives
this.searchInput.valueChanges.pipe(
  debounceTime(300),
  distinctUntilChanged(),
  switchMap(term => this.api.search(term))
).subscribe(results => this.results = results);
```

`concatMap` queues them (waits for each to finish, in order) — good for sequential writes where order must be preserved (don't want to cancel or interleave).

Q37. Memory leak in Angular — the classic subscription bug and fix.

```
// BUG: subscribes on init, never unsubscribes -> leaks when component destroyed & recreated repeatedly
ngOnInit() {
  this.dataService.data$.subscribe(d => this.data = d);
}

// FIX 1: async pipe (Angular manages subscribe/unsubscribe automatically)
// template: {{ data$ | async }}

// FIX 2: takeUntilDestroyed (Angular 16+) or manual Subject-based teardown
private destroy$ = new Subject<void>();
ngOnInit() {
  this.dataService.data$.pipe(takeUntil(this.destroy$)).subscribe(d => this.data = d);
}
ngOnDestroy() { this.destroy$.next(); this.destroy$.complete(); }
```

Q38. Angular dependency injection — `providedIn: 'root'` vs providing at module/component level? `providedIn: 'root'` = app-wide singleton, tree-shakeable if unused. Providing at a component level creates a **new instance per component instance** — useful for component-scoped state (e.g., a form-wizard service that shouldn't leak state between two open wizards).

Q39. Reactive Forms vs Template-Driven Forms — when would you pick each? Reactive: form model defined in TS, better for complex validation, dynamic forms, unit testing (no DOM needed). Template-driven: simpler forms, validation in template via directives — quicker for simple cases but harder to test and scale. Given Epicor is enterprise software, expect Reactive Forms to be the standard.

Q40. Explain Angular's `HttpInterceptor` — write one that attaches a JWT to every request.

```
@Injectable()
export class AuthInterceptor implements HttpInterceptor {
  constructor(private authService: AuthService) {}
  intercept(req: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {
    const token = this.authService.getToken();
    const authReq = token
      ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
      : req;
    return next.handle(authReq);
  }
}
```

Follow-up they may ask: how do you handle 401 responses globally? → catch in the interceptor, redirect to login or trigger silent token refresh.

Q41. Standalone components (Angular 15+) — what changed vs `NgModules`? Components can `import` their own dependencies directly (no `NgModule` declaration needed), simplifying lazy loading and reducing boilerplate. Root bootstrap uses `bootstrapApplication()` instead of `platformBrowserDynamic().bootstrapModule()`.

5B. React / TypeScript (in depth)

The JD is Angular-primary, but you've shipped production React (North Star Delivery Report, ResearchSwarm, JobHunterAgent) — worth being ready in case they ask you to compare frameworks or probe your broader front-end depth. This also doubles as prep for any React-based process you run in parallel.

Q1. Explain the React render cycle — what triggers a re-render, and what's a common wasted-render bug? A component re-renders when its own state changes, its props change (new reference), or its parent re-renders (by default, even if props didn't change — unlike Angular's `OnPush`, React re-renders children unconditionally unless you opt out). Classic wasted-render bug: passing a new inline object/array/function as a prop every render.

```
// BUG: new object literal every render -> React.memo on Child never helps, always re-renders
function Parent() {
  const [count, setCount] = useState(0);
  return <Child style={{ color: 'red' }} onClick={() => console.log('click')} />;
}

// FIX: stabilize references
function Parent() {
  const [count, setCount] = useState(0);
  const style = useMemo(() => ({ color: 'red' }), []);
  const handleClick = useCallback(() => console.log('click'), []);
  return <Child style={style} onClick={handleClick} />;
}
const Child = React.memo(function Child({ style, onClick }) { ... });
```

Q2. `useEffect` — explain the dependency array bugs (stale closures + missing cleanup).

```
// BUG 1: stale closure - count is captured at mount, never updates
useEffect(() => {
  const id = setInterval(() => console.log(count), 1000); // always logs 0
  return () => clearInterval(id);
}, []); // empty deps -> effect never re-runs, closure stays stale

// FIX: either add count to deps (effect re-runs each tick) or use functional update via a ref
useEffect(() => {
  const id = setInterval(() => setCount(c => c + 1), 1000);
  return () => clearInterval(id);
}, [count]);

// BUG 2: missing cleanup -> leaked subscription/listener across remounts
useEffect(() => {
  window.addEventListener('resize', handleResize);
  // no return -> listener never removed, duplicates on remount
}, []);

// FIX
useEffect(() => {
  window.addEventListener('resize', handleResize);
  return () => window.removeEventListener('resize', handleResize);
}, [handleResize]);
```

Q3. `useState` batching — what's the classic "stale state in a loop" bug?

```
// BUG: each setCount reads the same stale `count` from this render's closure -> only +1, not +3
function increment() {
  setCount(count + 1);
  setCount(count + 1);
  setCount(count + 1);
}

// FIX: functional updater reads the latest pending state, not the render's closure
function increment() {
  setCount(c => c + 1);
  setCount(c => c + 1);
  setCount(c => c + 1);
}
```

Note: React 18 batches updates even inside promises/timeouts/native event handlers (automatic batching) — worth mentioning if asked about React 18 changes.

Q4. `useMemo` vs `useCallback` — what's the actual difference? `useMemo` memoizes a **computed value**; `useCallback` memoizes a **function reference** (it's really `useMemo(() => fn, deps)` under the hood). Both exist purely to keep referential equality stable across renders so children wrapped in `React.memo` or effects with that value in their deps don't fire unnecessarily.

Q5. Custom hooks — write one that encapsulates a debounced search (mirrors your RxJS `switchMap` example on the Angular side).

```
function useDebouncedSearch(searchFn, delay = 300) {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);
  const latestRequestId = useRef(0);

  useEffect(() => {
    if (!query) { setResults([]); return; }
    const requestId = ++latestRequestId.current;
    const timer = setTimeout(async () => {
      const data = await searchFn(query);
      // guards against an older, slower request overwriting a newer one's results
      // (the React equivalent of switchMap cancellation)
      if (requestId === latestRequestId.current) setResults(data);
    }, delay);
    return () => clearTimeout(timer);
  }, [query, searchFn, delay]);

  return { query, setQuery, results };
}
```

Good talking point: React has no built-in cancellation primitive like RxJS's `switchMap`; you simulate “take latest” manually with a ref-based request ID or an `AbortController`.

Q6. Context API — when does it hurt performance, and how do you fix it? Every consumer of a Context re-renders whenever the Context value changes, **regardless of which part of the value it actually uses** — a single object passed as the value means unrelated state changes cause broad re-renders. Fix: split into multiple smaller contexts (e.g., `UserContext` and `ThemeContext` instead of one `AppContext`), or memoize the value object with `useMemo` so it doesn't change reference unless its contents actually changed.

Q7. Controlled vs uncontrolled components — trade-offs? Controlled: form value lives in React state, single source of truth, easy validation-as-you-type, but a re-render per keystroke. Uncontrolled: value lives in the DOM, read via `ref` on submit — fewer re-renders, less code for simple forms, but harder to do live validation. Libraries like `react-hook-form` use uncontrolled inputs internally for performance while still exposing a controlled-feeling API.

Q8. `useRef` — the two distinct use cases people conflate. 1) DOM access: `const inputRef = useRef(null); <input ref={inputRef} /> → inputRef.current.focus()`. 2) Mutable value that persists across renders **without** triggering a re-render when changed (unlike `useState`) — e.g., storing a previous prop value, a timer ID, or a “did we already fetch” flag.

Q9. Explain the Virtual DOM and reconciliation — what's the `key` prop bug in lists? React diffs the previous and new virtual DOM trees to compute the minimal real-DOM mutation. For lists, `key` tells React which element persisted vs which is new/removed/reordered.

```
// BUG: using array index as key when the list can reorder/filter
{items.map((item, index) => <Row key={index} data={item} />)}
// If item at index 0 is deleted, React thinks item 1 "became" item 0 and reuses its
// internal state/DOM node incorrectly -> wrong state attached to wrong row (classic bug
// with inputs inside lists retaining stale values after a delete/reorder).

// FIX: stable, unique id from the data itself
{items.map(item => <Row key={item.id} data={item} />)}
```

Q10. React vs Angular — how would you frame the comparison if asked directly in an Angular-shop interview?

Answer diplomatically and technically, not with a preference: Angular is a full framework (DI, routing, forms, RxJS, CLI conventions all built in) — good for large enterprise teams wanting consistency (arguably why Epicor standardized on it). React is a rendering library — more flexible, smaller core, but you assemble routing/state/forms from the ecosystem yourself. Mention you've shipped both in production (EMMA on Angular, North Star Delivery Report on React) so you can speak to trade-offs from direct experience rather than theory.

Q11. State management — when would you reach for something beyond Context/local state (Redux, Zustand, TanStack Query)? Local/Context state is fine for UI state (modals, form fields, theme). Server state (data fetched from an API) is a different problem — it needs caching, refetching, background revalidation, loading/error states per query. `TanStack Query` (React Query) solves that specifically, rather than shoving fetched data into Redux and hand-rolling cache invalidation. Good answer if your North Star Delivery Report used SSE/async data — mention how you handled loading/error/stale states there.

6. Testing (xUnit / Moq)

Q42. Write a unit test for a service method using xUnit + Moq.

```
public class OrderServiceTests
{
    [Fact]
    public async Task GetByIdAsync_ReturnsNull_WhenOrderNotFound()
    {
        // Arrange
        var mockRepo = new Mock<IOrderRepository>();
        mockRepo.Setup(r => r.GetByIdAsync(It.IsAny<int>()))
            .ReturnsAsync((Order?)null);
        var sut = new OrderService(mockRepo.Object);

        // Act
        var result = await sut.GetByIdAsync(99);

        // Assert
        Assert.Null(result);
        mockRepo.Verify(r => r.GetByIdAsync(99), Times.Once);
    }
}
```

Q43. [Fact] vs [Theory] in xUnit? [Fact] = single fixed test case. [Theory] + [InlineData(...)] = parameterized, runs the same test body against multiple inputs — reduces duplication for edge-case coverage (e.g., testing a discount calculator against several price/percentage combos).

Q44. How do you unit test a method that uses `DateTime.Now` internally? Don't call `DateTime.Now` directly inside logic — inject an abstraction (`IDateTimeProvider` / `TimeProvider` in .NET 8) so tests can control “now” deterministically. Otherwise tests become flaky/non-repeatable.

Q45. Mocking `DbContext` directly vs using an in-memory provider — trade-offs? Mocking `DbContext` / `DbSet` is brittle (LINQ operators like `.Include()` don't work well against mocked `DbSet`). Preferred: EF Core's **InMemory provider** or **SQLite in-memory** for integration-style tests that exercise real LINQ translation, reserving Moq for the repository/service *interfaces* in pure unit tests.

7. Cloud / DevOps (Azure, Docker, CI/CD)

Q46. Explain Azure Key Vault integration — why not just use appsettings.json for secrets? Config files can leak via source control, logs, or crash dumps. Key Vault centralizes secret storage, supports access via Managed Identity (no credentials in code at all), versioning, and audit logging. In ASP.NET Core: `builder.Configuration.AddAzureKeyVault(...)` merges vault secrets into the standard `IConfiguration` pipeline.

Q47. Managed Identity — what problem does it solve? Removes the “secret to get a secret” chicken-and-egg problem — an Azure resource (App Service, AKS pod) gets an identity trusted by Azure AD directly, so it can call Key Vault/Storage/SQL without any stored credential at all.

Q48. Explain a basic CI/CD pipeline stage breakdown you’d expect in Azure Pipelines. Build (restore, compile, run unit tests, fail fast on red tests) → Artifact publish → Release (deploy to Dev slot → run smoke tests → swap to Staging → manual approval gate → swap to Production). Blue-green via **deployment slots** avoids downtime.

Q49. Docker — what’s the difference between an image and a container, and why multi-stage builds? Image = the built, immutable template; container = a running instance of that image. Multi-stage builds use a heavier SDK image to *compile* the app, then copy only the compiled output into a slim runtime image — final image is much smaller and doesn’t ship the SDK/compiler in production.

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY . .
RUN dotnet publish -c Release -o /app

FROM mcr.microsoft.com/dotnet/aspnet:8.0
WORKDIR /app
COPY --from=build /app .
ENTRYPOINT ["dotnet", "MyApp.dll"]
```

Q50. Kubernetes basics — what’s a Pod, a Deployment, and a Service? Pod = smallest deployable unit, one or more containers sharing network/storage. Deployment = manages Pod replicas, rolling updates, self-healing (restarts crashed pods). Service = stable network endpoint/load balancer in front of a changing set of Pod IPs.

8. System Design / Architecture Scenario (likely for a 3-5 yr band)

Q51. “How would you design a feature-modernization of a legacy .NET app without a full rewrite?” (directly from JD: “modernizing legacy applications”) Strangler Fig pattern: put a facade/API gateway in front of the legacy app, route new features to new .NET 8 services incrementally, migrate one bounded module at a time, keep the legacy system running until every capability has been replaced. Emphasize: risk reduction, no big-bang cutover, ability to roll back per-module.

Q52. “A production API is suddenly slow under load — walk me through your diagnosis.” Structured answer: 1) Check dashboards/App Insights for response time, CPU, memory, DB DTU/connection pool exhaustion. 2) Check recent deploys (regression?). 3) Query the DB’s execution plans for the slow endpoints’ queries. 4) Check for N+1 queries, missing indexes, thread pool starvation (blocking async calls). 5) Check connection pool exhaustion from `DbContext` not being disposed properly. 6) Mitigate first (scale out / restart), root-cause after.

9. Behavioral (STAR) — tie to your actual projects

Given your resume's Agentic AI sprint analytics platform and North Star Delivery Report, prep 2-3 STAR stories: - **A time you improved performance:** SQL Server stored procedure optimization work — quantify (you have real numbers from your EMMA365 optimization work — cite the actual before/after if asked). - **A time you migrated/replaced a legacy system:** your Duende IdentityServer → Microsoft Entra ID migration — great answer for “modernizing legacy applications.” - **A time you worked cross-functionally:** North Star Delivery Report consolidating Azure DevOps/GitHub/pipeline/cost data for programme leadership. - **Handling ambiguity/tight deadlines:** Agile ceremonies, sprint planning — the JD explicitly lists this.

Anticipate: “You’re at 3 years, we asked for 3-5 — why should we consider you?” Answer with production ownership breadth (you’ve owned Azure Front Door migration, RBAC design, Key Vault, AKS/Helm/Docker deployment — that’s senior-leaning scope for the tenure) rather than defending the number itself.

9B. Core Fundamentals Deep Dive — OOP, Async/Await, JavaScript, Closures, Promises, Azure, Kubernetes, Docker

This section is a from-scratch basics-to-depth pass on exactly what you asked for. Treat it as your “can’t get caught out” list — these are the questions that sound simple but trip people up when asked to explain *why*, not just *what*.

OBJECT-ORIENTED PROGRAMMING

Q1. The four pillars — define each with a one-line code example.

```
// 1. Encapsulation: hide internal state, expose controlled access
public class Account
{
    private decimal _balance; // hidden
    public decimal Balance => _balance; // read-only exposure
    public void Deposit(decimal amount) { if (amount > 0) _balance += amount; }
}

// 2. Abstraction: expose *what*, hide *how*
public interface IPaymentProcessor { Task<bool> ChargeAsync(decimal amount); }
// Caller doesn't care if it's Stripe, Razorpay, or a bank API underneath.

// 3. Inheritance: reuse + specialize
public class Employee { public string Name { get; set; } public virtual decimal CalculatePay() => 0; }
public class SalariedEmployee : Employee { public decimal Salary { get; set; } public override decimal CalculatePay()
Salary; }

// 4. Polymorphism: same call, different behavior at runtime
Employee e = new SalariedEmployee { Salary = 50000 };
e.CalculatePay(); // resolves to SalariedEmployee's override at runtime (virtual dispatch)
```

Q2. Compile-time (method overloading) vs runtime (method overriding) polymorphism — bug to watch for?

```
public class Base { public virtual void Print() => Console.WriteLine("Base"); }
public class Derived : Base {
    public new void Print() => Console.WriteLine("Derived"); // 'new' hides, doesn't override!
}

Base b = new Derived();
b.Print(); // prints "Base" — surprising! 'new' breaks virtual dispatch because
           // the static (compile-time) type Base is used, not the runtime type.
// Fix: use 'override' on a 'virtual' base method if polymorphic dispatch is intended.
```

Q3. Composition over inheritance — why, with an example? Deep inheritance chains get fragile (the “fragile base class” problem — changing a base class breaks distant subclasses unexpectedly). Composition builds behavior by injecting smaller collaborators instead:

```
// Inheritance-heavy (fragile): Bird -> FlyingBird -> Duck, but what about a Penguin?
// Composition: inject only the capability that applies
public interface IFlyBehavior { void Fly(); }
public class Duck { private readonly IFlyBehavior _flyBehavior; public Duck(IFlyBehavior f) => _flyBehavior = f; publ
void PerformFly() => _flyBehavior.Fly(); }
public class Penguin { /* no IFlyBehavior injected at all — no awkward override-to-throw */ }
```

Q4. Interface vs abstract class — when do you pick one over the other? Abstract class: shared state + some default implementation, “is-a” relationship, single inheritance only. Interface: pure contract, multiple interfaces can be implemented, “can-do” capability. Rule of thumb: if unrelated classes need to share a capability (e.g., `IDisposable`, `IComparable`), interface. If a family of related types shares real code, abstract class.

Q5. What is the Liskov Substitution Principle violation in this classic example — and the fix?


```
// BUG: Square "is-a" Rectangle mathematically, but violates LSP as a class hierarchy
public class Rectangle { public virtual int Width { get; set; } public virtual int Height { get; set; } }
public class Square : Rectangle
{
    public override int Width { set { base.Width = base.Height = value; } }
    public override int Height { set { base.Width = base.Height = value; } }
}
// Code that does rect.Width = 5; rect.Height = 10; expecting Area == 50
// silently breaks when 'rect' is actually a Square (Area becomes 100).

// FIX: don't force an inheritance relationship where behavior contracts diverge -
// use a shared IShape interface with independent implementations instead.
```

JAVASCRIPT FUNDAMENTALS

Q6. `var` vs `let` vs `const` — the classic loop closure bug.

```
// BUG: var is function-scoped, not block-scoped - all 3 callbacks share the SAME i
for (var i = 0; i < 3; i++) {
    setTimeout(() => console.log(i), 100); // prints 3, 3, 3
}

// FIX: let is block-scoped - a NEW binding of i is created per iteration
for (let i = 0; i < 3; i++) {
    setTimeout(() => console.log(i), 100); // prints 0, 1, 2
}
```

Q7. `==` vs `===` — a bug caused by type coercion.

```
console.log(0 == ''); // true (coercion: '' -> 0)
console.log(0 === ''); // false (no coercion, different types)
console.log(null == undefined); // true
console.log(null === undefined); // false
// Rule: always use === / !== unless you have a specific, documented reason for coercion.
```

Q8. `this` binding — explain the 4 ways `this` is determined, with a common bug.

```
const obj = {
    name: 'Order',
    items: [1, 2, 3],
    printItems() {
        // BUG: regular function loses 'this' - inside forEach callback, 'this' is undefined (strict mode) or global
        this.items.forEach(function (item) {
            console.log(this.name, item); // TypeError: Cannot read properties of undefined
        });
    }
};

// FIX 1: arrow function inherits 'this' lexically from enclosing scope
printItemsFixed() {
    this.items.forEach((item) => console.log(this.name, item));
}

// FIX 2: bind explicitly
this.items.forEach(function (item) { console.log(this.name, item); }.bind(this));
```

The 4 rules, ranked by precedence: `new` binding > explicit (`call` / `apply` / `bind`) > implicit (`obj.method()`) > default (global/undefined in strict mode). Arrow functions ignore all 4 — they capture `this` from where they're *defined*, not called.

Q9. Hoisting — what actually gets hoisted, and the Temporal Dead Zone bug.

```
console.log(a); // undefined (var is hoisted AND initialized to undefined)
var a = 5;

console.log(b); // ReferenceError: Cannot access 'b' before initialization (TDZ)
let b = 10;
// let/const ARE hoisted to the top of the block, but stay uninitialized until
// their declaration line executes - that gap is the "Temporal Dead Zone".
```

Q10. Event loop, call stack, microtasks vs macrotasks — predict the output.

```

console.log('1');
setTimeout(() => console.log('2'), 0); // macrotask queue
Promise.resolve().then(() => console.log('3')); // microtask queue
console.log('4');

// Output: 1, 4, 3, 2
// Reasoning: synchronous code runs first (1, 4). Then ALL microtasks (Promise .then)
// drain before the next macrotask (setTimeout) is even considered – even a
// setTimeout(fn, 0) waits behind every pending microtask.

```

Q11. Prototypal inheritance — how does JS “class” syntax actually work under the hood?

```

class Animal {
  constructor(name) { this.name = name; }
  speak() { console.log(`${this.name} makes a sound`); }
}
class Dog extends Animal {
  speak() { console.log(`${this.name} barks`); }
}
// Under the hood: Dog.prototype.__proto__ === Animal.prototype
// 'class' is syntactic sugar over JS's existing prototype chain – there's no
// separate "class" mechanism in the engine, method lookup walks the prototype chain.

```

CLOSURES

Q12. Define a closure and show the classic “private counter” pattern. A closure is a function that retains access to variables from its enclosing lexical scope even after that outer function has returned.

```

function createCounter() {
  let count = 0; // this variable is "closed over"
  return {
    increment: () => ++count,
    decrement: () => --count,
    getValue: () => count
  };
}
const counter = createCounter();
counter.increment(); counter.increment();
console.log(counter.getValue()); // 2
// 'count' is fully private – no way to access it except through the returned methods.

```

Q13. Closures-in-a-loop bug (the classic interview question) — show it with `var` AND the two fixes.

```

// BUG
function createFunctions() {
  var fns = [];
  for (var i = 0; i < 3; i++) {
    fns.push(function () { console.log(i); });
  }
  return fns;
}
createFunctions().forEach(fn => fn()); // 3, 3, 3 – all closures share the one 'i'

// FIX 1: let (block-scoped, new binding per iteration)
for (let i = 0; i < 3; i++) { fns.push(() => console.log(i)); } // 0, 1, 2

// FIX 2: IIFE to create a new scope per iteration (the pre-ES6 way)
for (var i = 0; i < 3; i++) {
  (function (capturedI) { fns.push(() => console.log(capturedI)); })(i);
}

```

Q14. Memory leak risk with closures — how can they cause one? A closure keeps its entire enclosing scope alive as long as the closure itself is reachable — including large objects you didn’t mean to retain.

```

function setup() {
  const hugeData = new Array(1_000_000).fill('x'); // large object
  const smallHelper = () => console.log('done');
  return smallHelper; // hugeData is NOT used by smallHelper, but stays in scope's closure
  // if the JS engine can't prove hugeData is unused (older/less-optimizing engines),
  // it may be kept alive as long as smallHelper is referenced somewhere.
}

```

Practical takeaway: be deliberate about what large data structures live inside a closure that outlives its creator (e.g., an event handler registered once but referencing a big dataset).

PROMISES & ASYNC/AWAIT

Q15. Promise states — pending/fulfilled/rejected — and the classic “unhandled rejection” bug.

```
// BUG: no .catch(), rejection is silently swallowed / logged as unhandled
fetchUser(id).then(user => console.log(user.name));
// If fetchUser rejects, Node/browser logs "UnhandledPromiseRejection" – the error
// is lost, no graceful handling, and in Node this can even crash the process.

// FIX
fetchUser(id)
  .then(user => console.log(user.name))
  .catch(err => console.error('Failed to fetch user:', err));
```

Q16. Promise.all VS Promise.allSettled VS Promise.race VS Promise.any — with a bug from picking the wrong one.

```
// BUG: Promise.all fails FAST – if ANY promise rejects, you lose all other results
const results = await Promise.all([fetchA(), fetchB(), fetchC()]);
// if fetchB() rejects, you get NOTHING from fetchA/fetchC even though they succeeded

// FIX: allSettled always resolves, giving you status per-promise
const results = await Promise.allSettled([fetchA(), fetchB(), fetchC()]);
results.forEach(r => r.status === 'fulfilled' ? console.log(r.value) : console.error(r.reason));

// race: resolves/rejects as soon as the FIRST promise settles (useful for timeouts)
const result = await Promise.race([fetchData(), timeout(5000)]);

// any: resolves as soon as the FIRST one FULFILLS (ignores rejections unless all reject)
const first = await Promise.any([mirror1(), mirror2(), mirror3()]);
```

Q17. Sequential vs parallel awaits — a real performance bug.

```
// BUG: sequential when they don't depend on each other – wastes time (3x latency)
async function loadDashboard() {
  const user = await fetchUser(); // waits ~200ms
  const orders = await fetchOrders(); // then waits another ~200ms
  const stats = await fetchStats(); // then another ~200ms
  return { user, orders, stats }; // total: ~600ms
}

// FIX: fire independent requests in parallel
async function loadDashboard() {
  const [user, orders, stats] = await Promise.all([
    fetchUser(), fetchOrders(), fetchStats()
  ]);
  return { user, orders, stats }; // total: ~200ms (bounded by the slowest one)
}
```

Q18. async function return value — what does it actually return, and a common misunderstanding? An async function ALWAYS returns a Promise, even if you return a plain value inside it — the value is auto-wrapped.

```
async function getValue() { return 42; }
getValue(); // returns Promise<42>, NOT 42 directly
getValue().then(v => console.log(v)); // 42

// BUG: forgetting this and trying to use the value synchronously
const value = getValue();
console.log(value + 1); // "[object Promise]1" – string concatenation bug, not 43
```

Q19. Error handling in async/await — try/catch scope bug.

```
// BUG: try/catch only wraps the await, but a .then() chain elsewhere is unguarded,
// OR: forgetting that a non-awaited promise inside try won't be caught by that try
async function process() {
  try {
    doSomethingAsync(); // NOT awaited! If it rejects, this catch will NOT catch it
  } catch (err) {
    console.error('caught:', err); // never runs for the unawaited rejection above
  }
}

// FIX: always await promises you want try/catch to actually cover
async function process() {
  try {
    await doSomethingAsync();
  } catch (err) {
    console.error('caught:', err);
  }
}
```

Q20. Write an async function with retry logic and exponential backoff (a realistic senior-level ask).

```
async function fetchWithRetry(fn, retries = 3, delayMs = 500) {
  for (let attempt = 1; attempt <= retries; attempt++) {
    try {
      return await fn();
    } catch (err) {
      if (attempt === retries) throw err; // exhausted retries, rethrow
      const backoff = delayMs * Math.pow(2, attempt - 1); // exponential
      await new Promise(resolve => setTimeout(resolve, backoff));
    }
  }
}
```

AZURE (mapped to JD: App Services, Azure SQL, Storage, Functions, Key Vault)

Q21. Azure App Service — Deployment Slots, why they matter. Slots let you deploy to a “staging” instance with its own hostname, warm it up, run smoke tests, then **swap** with production — a near-zero-downtime deployment (swap is a VIP swap, not a redeploy). Rolling back is just swapping back.

Q22. Azure Functions — trigger types and a cold start consideration. Triggers: HTTP, Timer (cron-like), Queue (Service Bus/Storage Queue), Blob (on file upload), Event Grid. Consumption plan = pay-per-execution but has **cold starts** (first invocation after idle spins up a new instance, adds latency) — Premium plan keeps instances warm for latency-sensitive workloads.

Q23. Azure Storage — Blob vs Table vs Queue, when to use which? Blob = unstructured files (images, documents, backups) — hot/cool/archive tiers for cost optimization. Table = NoSQL key-value, cheap, simple partition-key/row-key lookups (no complex queries/joins). Queue = simple message passing between services, decoupling producer from consumer (vs Service Bus, which adds topics/subscriptions, dead-lettering, sessions for more complex messaging).

Q24. Azure Key Vault + Managed Identity — full flow (you’ve built this — be ready to narrate it fluently). App Service/AKS pod is assigned a Managed Identity (system- or user-assigned) → Key Vault access policy (or RBAC role) grants that identity `get / list` secret permissions → app code calls `builder.Configuration.AddAzureKeyVault(vaultUri, new DefaultAzureCredential())` → secrets merge into `IConfiguration` at startup, no credential ever stored in code or config files.

Q25. Azure SQL — DTU vs vCore purchasing model? DTU (Database Transaction Unit): bundled compute+storage+IO in preset tiers, simpler for smaller/predictable workloads. vCore: independently scale compute and storage, choose your own hardware generation, better cost transparency for larger workloads, supports Azure Hybrid Benefit (bring your own SQL Server license for savings).

Q26. Explain Azure Front Door (you led this migration) — Classic vs Premium, in your own words for the interview. Front Door is a global entry point (CDN + reverse proxy + WAF) routing users to the closest healthy backend. Premium adds tighter WAF rule management, Private Link support to backends (no public backend exposure), and managed rule sets updated by Microsoft — worth mentioning the *security* driver (WAF policy enforcement) alongside the performance driver (routing) since that’s likely why the migration mattered to the business.

KUBERNETES (AKS) — deeper than the basics section above

Q27. ConfigMaps vs Secrets — why are they separate objects if they’re used similarly? Both inject config into Pods (as env vars or mounted files), but Secrets are base64-encoded (not encrypted by default at rest unless you enable encryption-at-rest / integrate Key Vault via CSI driver) and access can be restricted via RBAC separately from ConfigMaps — keeping the security boundary distinct even though the mechanism looks similar.

Q28. Liveness vs Readiness probes — a bug from confusing them.

```

livenessProbe:
  httpGet: { path: /healthz, port: 8080 }
  periodSeconds: 10
readinessProbe:
  httpGet: { path: /ready, port: 8080 }
  periodSeconds: 5

```

Liveness failing → Kubernetes **kills and restarts** the container (it thinks the app is deadlocked/crashed). Readiness failing → Kubernetes just **stops routing traffic** to that Pod, without restarting it. Bug: pointing both at the same slow endpoint (e.g., one that checks DB connectivity) — a slow DB makes Kubernetes think the app itself crashed and restart it repeatedly, when really it just isn't ready yet. Fix: liveness checks "is the process alive" (cheap, no external calls); readiness checks "can it serve traffic right now" (can include dependency checks).

Q29. Horizontal Pod Autoscaler (HPA) — how does it decide to scale? Polls metrics (CPU/memory by default, or custom metrics via an adapter) against a target utilization you set (e.g., 70% CPU) and adjusts replica count within `minReplicas` / `maxReplicas` bounds. Requires resource `requests` to be set on the Pod spec — without a request value, HPA has no baseline to calculate percentage utilization against.

Q30. Rolling update vs recreate deployment strategy? Rolling update (default): gradually replaces old Pods with new ones, keeping some capacity available throughout — zero downtime if done right (paired with readiness probes so traffic isn't sent to a not-yet-ready new Pod). Recreate: kills all old Pods first, then starts new ones — causes downtime, only used when the app can't tolerate two versions running simultaneously (e.g., a schema-incompatible DB migration mid-rollout).

DOCKER — deeper

Q31. Docker layer caching — why does instruction order in a Dockerfile matter for build speed?

```

# BAD: copying everything before restoring dependencies invalidates the cache
# on every single source code change, forcing a full package re-download every build
COPY . .
RUN dotnet restore
RUN dotnet build

# GOOD: copy only the project file first — this layer only invalidates when
# dependencies actually change, not on every code edit
COPY *.csproj ./
RUN dotnet restore
COPY . .
RUN dotnet build

```

Q32. Why avoid running containers as root — and how do you fix it? Root inside a container, if a container-escape vulnerability exists, has a much larger blast radius on the host. Fix:

```

FROM mcr.microsoft.com/dotnet/aspnet:8.0
RUN adduser --disabled-password --gecos '' appuser
USER appuser
COPY --chown=appuser:appuser . .

```

Q33. CMD vs ENTRYPOINT — what's the practical difference? `ENTRYPOINT` defines the fixed executable that always runs; `CMD` provides default *arguments* to it (overridable at `docker run`). Combining them is the common pattern: `ENTRYPOINT ["dotnet", "MyApp.dll"]` with `CMD ["--environment=Production"]` lets you override just the environment at runtime without redefining the whole `entrypoint`.

Q34. Docker networking basics — bridge vs host vs none? Bridge (default): container gets its own isolated network namespace + virtual NIC, ports must be explicitly published (`-p 8080:80`) to be reachable from the host. Host: container shares the host's network stack directly (no port mapping needed, but no isolation — used rarely, mostly for performance-sensitive or network-tooling containers). None: no networking at all, for isolated batch jobs.

10. Quick-Fire Round (rapid recall, likely early screening questions)

Question	One-liner answer
<code>abstract class</code> vs <code>interface</code> ?	Abstract class can have implementation + state; interface (pre-C# 8) is pure contract. C# 8+ interfaces can have default methods, blurring this.
<code>IEnumerable</code> vs <code>List</code> ?	<code>IEnumerable</code> is a forward-only, lazy contract; <code>List<T></code> is a concrete, indexable, in-memory collection.
<code>out</code> vs <code>ref</code> ?	<code>out</code> doesn't need to be initialized before the call, must be assigned inside; <code>ref</code> must be initialized before, can be read/written.
<code>Task</code> vs <code>Task<T></code> ?	<code>Task</code> = async void-return; <code>Task<T></code> = async method returning a value.
<code>is</code> vs <code>as</code> ?	<code>as</code> returns null on failed cast (no exception); <code>is</code> returns bool; <code>is</code> pattern matching (<code>if (obj is Order o)</code>) combines both.
Primary constructor (C# 12)?	<code>public class Order(int id, string name) { }</code> — shorthand constructor syntax reducing boilerplate.
<code>record</code> vs <code>class</code> ?	<code>record</code> gives value-based equality + <code>with</code> non-destructive mutation, good for DTOs.
Connection pooling in SQL Server?	ADO.NET/EF Core reuses physical connections from a pool keyed by connection string — avoids the cost of opening a new TCP+auth handshake per query.
<code>HttpClient</code> — why not <code>new HttpClient()</code> per request?	Socket exhaustion — use <code>IHttpClientFactory</code> (typed clients) which manages the connection pool lifetime correctly.

How to use this in the next 4 days

1. **Day 1-2:** Re-derive the bug-spotting examples above from memory without looking, out loud, as if explaining to an interviewer.
2. **Day 2-3:** Practice writing the stored procedure, LINQ GroupBy, and the Web API controller from scratch on a blank editor — no autocomplete.
3. **Day 3:** Rehearse your 3 STAR stories against your actual resume metrics — have the numbers ready (97.8% accuracy, the SQL optimization gains).
4. **Day 4 (interview day):** Skim this doc once in the morning; don't cram new material the day of.

Good luck — you've got real production depth here (Entra ID migration, AKS/Helm, Key Vault, Front Door) that punches above 3 years of tenure. Lead with that when they probe the experience gap.